

Universidad Rafael Landívar.
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas.
Estructura de Datos II
Ing. Manuel Rojas



PROYECTO DE LABORATORIO NO. 1

Santiago Jéfté Batz Rodríguez - 1548725

Quetzaltenango, 8 de septiembre de 2026

Descripcion del Proyecto:

El proyecto consiste en un sistema de gestión de biblioteca, y el programa permite registrar, buscar y eliminar libros, así como gestionar préstamos y devoluciones, mostrar el catálogo ordenado por título y consultar información relacionada con la cantidad de préstamos y disponibilidad de los libros.

Cada libro se identifica mediante un código único y contiene información como título, autor, categoría, copias disponibles y cantidad de veces que ha sido prestado.

Para organizar la información se implementaron tres estructuras de datos: un Árbol B+, un Max Heap y un Min Heap, donde el Árbol B+ se utiliza principalmente para buscar libros mediante su código, el Max Heap permite identificar el libro con mayor cantidad de préstamos y el Min Heap permite identificar el libro con menor cantidad de copias disponibles.

La clase Biblioteca se encarga de coordinar las operaciones entre las diferentes estructuras para mantener la información actualizada.

Estructura del proyecto:

```
Proyecto Biblioteca/
|
|—— Program.cs
|—— Modelos/
|   |—— Libro.cs
|—— Estructuras/
|   |—— ArbolBPlus.cs
|   |—— MaxHeap.cs
|   |—— MinHeap.cs
|—— Servicios/
|   |—— Biblioteca.cs
|   |—— GestorArchivos.cs
|—— Datos/
|   |—— libros.csv
```

Program.cs: es el punto de entrada del programa, contiene el menú principal de consola, recibe las opciones ingresadas por el usuario y solicita los datos necesarios para ejecutar cada operación.

Desde este archivo se utilizan los métodos proporcionados por la clase Biblioteca, evitando colocar directamente en el menú la lógica interna de las estructuras de datos.

Libro.cs: contiene la clase Libro, utilizada para representar cada libro almacenado en el sistema, donde entre sus propiedades se encuentran: código, título, autor, categoría, copias disponibles, copias totales, cantidad de veces prestado.

Aca la propiedad CopiasTotales se utiliza para controlar que una devolución no aumente la cantidad de copias disponibles por encima de la cantidad registrada

ArbolBPlus.cs: contiene la implementación propia del Árbol B+, y esta estructura utiliza el código del libro como clave y permite realizar operaciones como: inserción, búsqueda, eliminación, recorrido, impresión.

Dentro de esta clase también se encuentra la clase privada Nodo, utilizada para representar los nodos internos y las hojas del árbol.

MaxHeap.cs: implementa el Max Heap mediante un arreglo de objetos Libro, y su prioridad corresponde a la propiedad VecesPrestado, es por esta razón, el libro ubicado en la raíz del Max Heap es el que posee la mayor cantidad de préstamos. La clase contiene operaciones de inserción, eliminación, búsqueda, reorganización e impresión.

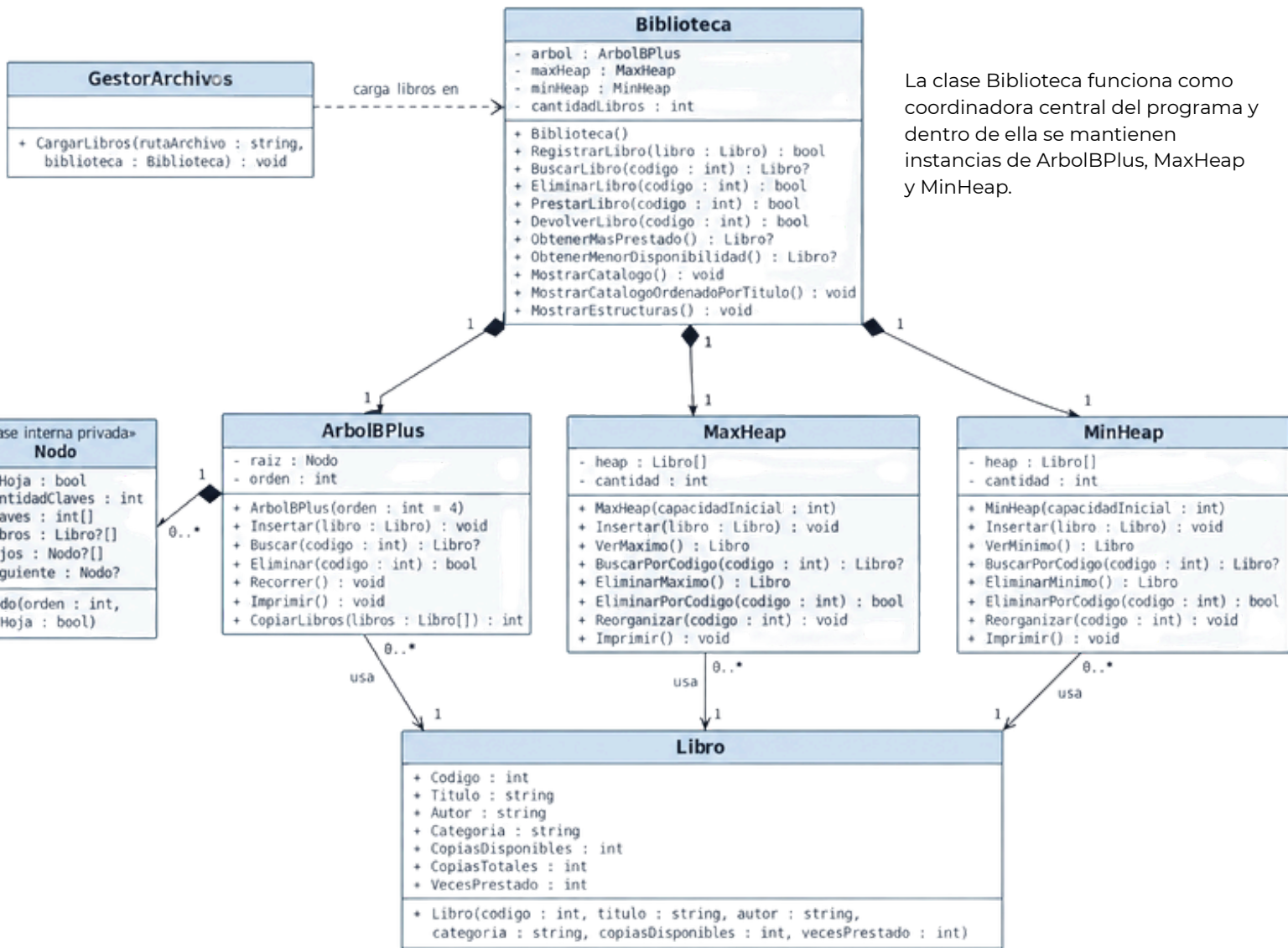
MinHeap.cs: implementa el Min Heap mediante un arreglo de objetos Libro. En este caso, la prioridad corresponde a CopiasDisponibles, como resultado, el libro ubicado en la raíz es el que posee menor cantidad de copias disponibles. También contiene operaciones de inserción, eliminación, búsqueda, reorganización e impresión.

Biblioteca.cs: es la clase encargada de coordinar las operaciones principales del sistema, el cual mantiene referencias a: ArbolBPlus, MaxHeap, MinHeap

Cuando se registra o elimina un libro, esta clase realiza los cambios necesarios en las tres estructuras para mantener la información sincronizada, también contiene la lógica correspondiente a préstamos, devoluciones, consultas y presentación del catálogo.

GestorArchivos.cs: es responsable de cargar los datos iniciales desde el archivo libros.csv, ya que comprueba que el archivo exista, lee sus líneas, convierte los datos numéricos y crea los objetos Libro correspondientes, donde después de crear cada libro, utiliza el método RegistrarLibro de la clase Biblioteca para incorporarlo al sistema.

libros.csv: contiene los datos iniciales utilizados por el sistema, cada registro posee la siguiente información: codigo, titulo, autor ,categoria, copiasDisponibles, vecesPrestado, y así de esta forma, la información inicial no se encuentra escrita directamente dentro del código fuente.

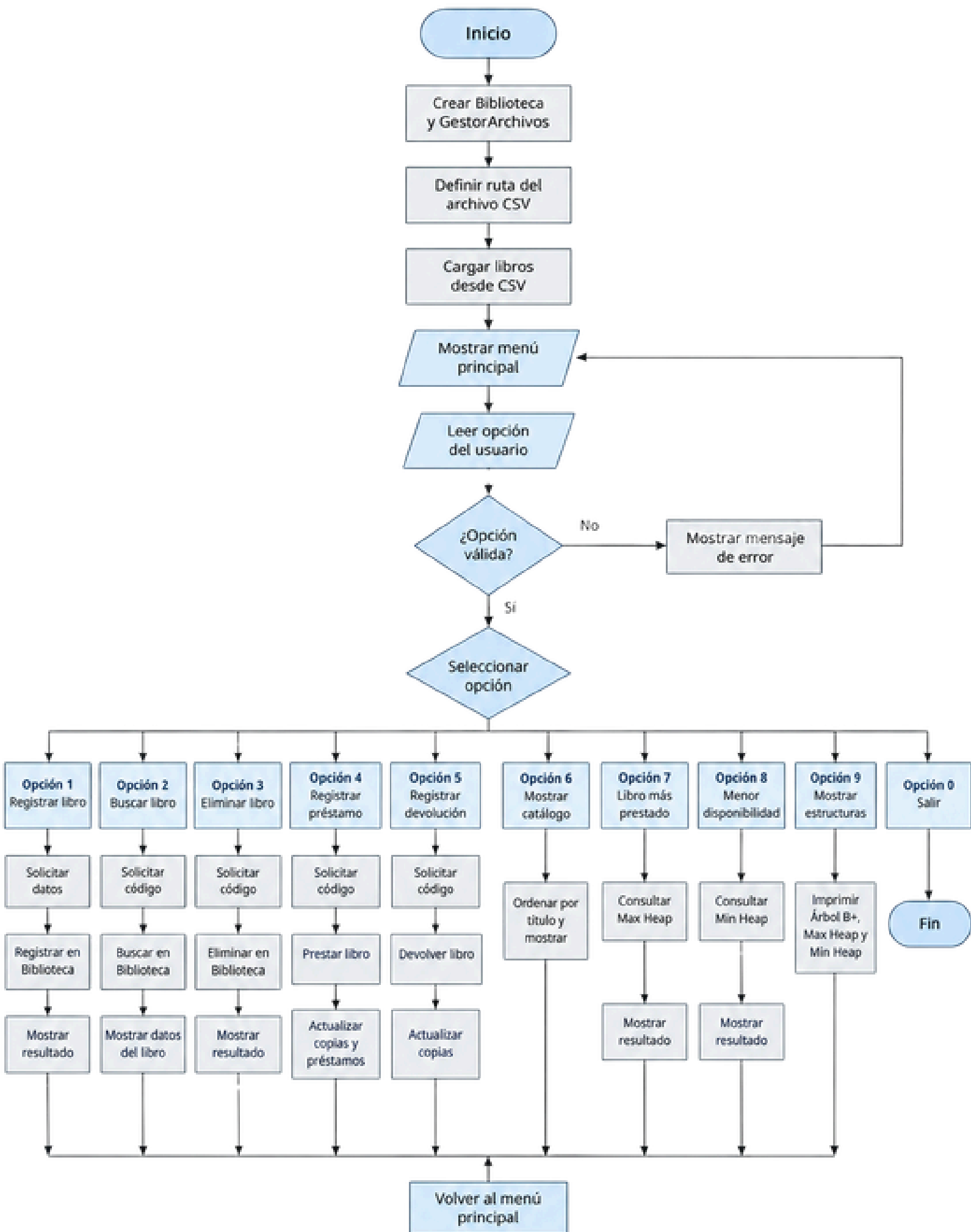


El diagrama de clases representa la organización principal del sistema y las relaciones existentes entre sus componentes

Estas tres estructuras trabajan con objetos de tipo Libro, aunque cada una utiliza diferentes propiedades para organizar la información:

- ArbolBPlus utiliza el código del libro como clave.
- MaxHeap utiliza la cantidad de veces que el libro ha sido prestado.
- MinHeap utiliza la cantidad de copias disponibles.

La clase GestorArchivos depende de Biblioteca, debido a que después de leer los datos provenientes del archivo CSV utiliza la biblioteca para registrar cada libro, y en el caso del Árbol B+, se utiliza además una clase interna privada llamada Nodo, la cual esta representa cada nodo del árbol y almacena las claves, referencias a libros, hijos y el enlace hacia la siguiente hoja. La separación de las clases permite que cada una tenga una responsabilidad específica, evitando concentrar toda la lógica del sistema dentro de un solo archivo.



La ejecución inicia creando los objetos necesarios para trabajar con la biblioteca y la carga de archivos, posteriormente se establece la ruta del archivo CSV y se cargan los libros registrados en él, donde una vez finalizada la carga inicial, el programa presenta el menú principal y solicita al usuario una opción, si la entrada ingresada no es válida, se muestra un mensaje de error y se vuelve a presentar el menú. Cuando la opción es válida, el programa ejecuta la operación correspondiente mediante una estructura switch, donde al finalizar una operación, el programa regresa nuevamente al menú principal, permitiendo ejecutar otra acción.

Este ciclo continúa hasta que el usuario selecciona la opción 0, momento en el cual finaliza la ejecución del programa, y el flujo de las operaciones. para las operaciones que requieren identificar un libro, el usuario proporciona su código, y la clase Biblioteca utiliza el Árbol B+ para localizar el libro y posteriormente realiza la operación correspondiente.

Por ejemplo, durante un préstamo:

Código → Buscar libro en Árbol B+ → Verificar disponibilidad → Disminuir copias disponibles → Aumentar cantidad de préstamos → Reorganizar Max Heap → Reorganizar Min Heap

En una devolución únicamente se incrementa la cantidad de copias disponibles y se reorganiza el Min Heap, ya que la cantidad acumulada de préstamos no disminuye, cuando se elimina un libro, la operación se realiza en el Árbol B+, Max Heap y Min Heap para evitar que queden referencias a un libro que ya no pertenece al catálogo.

Implementación del Árbol B+

El Árbol B+ se implementó en la clase ArbolBPlus y se utiliza para organizar los libros mediante su propiedad Codigo, en el sistema se crea el árbol con orden 4: `arbol = new ArbolBPlus(4)`, esto significa que cada nodo puede almacenar como máximo tres claves y, en el caso de los nodos internos, hasta cuatro referencias hacia nodos hijos.

Para representar los nodos se utiliza una clase interna llamada Nodo. Cada nodo almacena la siguiente información:

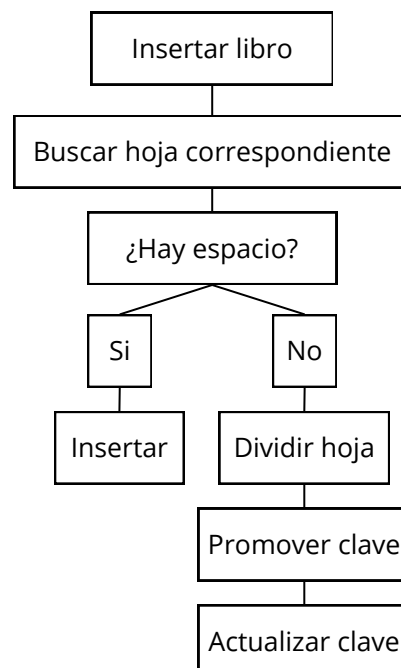
- si corresponde a una hoja o a un nodo interno;
- cantidad actual de claves;
- arreglo de claves;
- arreglo de libros;
- arreglo de hijos;
- referencia a la siguiente hoja.

La propiedad Siguiente permite enlazar las hojas entre sí y realizar un recorrido secuencial de los libros almacenados.

Inserción: para insertar un libro, primero se verifica que su código no se encuentre registrado, posteriormente se busca la hoja correspondiente, si esta posee espacio disponible, el libro se inserta manteniendo las claves ordenadas. Cuando una hoja se encuentra llena, se realiza una división, los elementos se distribuyen entre la hoja original y una nueva hoja, y la primera clave de la hoja derecha se utiliza como clave separadora para el nodo padre.

Si un nodo interno también se llena, este proceso de división puede continuar hacia niveles superiores, en caso de que la raíz se divida, se crea una nueva raíz.

Se puede ver como:



Búsqueda: la búsqueda utiliza el código del libro, desde la raíz se comparan las claves almacenadas para determinar por cuál hijo debe continuar el recorrido, este procedimiento se repite hasta alcanzar una hoja, donde una vez encontrada la hoja correspondiente, se recorren sus claves para localizar el código solicitado y devolver el objeto Libro, viendo este método es utilizado por Biblioteca para operaciones como búsqueda, préstamo, devolución y eliminación.

Eliminación: para eliminar un libro se localiza primero la hoja donde se encuentra su código, después de eliminarlo, se comprueba si el nodo conserva la cantidad mínima de elementos necesaria, ya cuando un nodo queda con menos elementos de los permitidos, el código intenta mantener la estructura mediante:

- redistribución de elementos con un nodo hermano
- fusión con un nodo hermano
- actualización de las claves separadoras del padre

El árbol también contempla el caso en que la raíz queda sin claves y puede ser reemplazada por uno de sus hijos.

Recorrido: para recorrer los libros, el método Recorrer() baja primero hasta la hoja situada más a la izquierda, posteriormente utiliza la referencia Siguiente de cada hoja para avanzar hasta llegar al final.

Hoja 1 → Hoja 2 → Hoja 3 → ... → null

De esta forma se pueden recorrer los registros almacenados sin volver a subir por los nodos internos.

Implementación del Max Heap

El Max Heap se implementó en la clase MaxHeap mediante un arreglo de objetos Libro

```
private Libro[] heap;  
private int cantidad;
```

La propiedad utilizada como prioridad es: VecesPrestado, por lo tanto, el libro con mayor cantidad de préstamos se mantiene en la posición inicial del arreglo: heap[0] Esto permite obtener directamente el libro más prestado mediante el método VerMaximo().

Representación del Heap: al utilizar un arreglo, las posiciones de padre e hijos se calculan de la siguiente forma:

Padre = (indice - 1) / 2

Hijo izquierdo = 2 * indice + 1

Hijo derecho = 2 * indice + 2

No fue necesario implementar nodos enlazados para el Heap.

Inserción: un nuevo libro se coloca inicialmente al final del arreglo, después se ejecuta el método Subir(), que compara el número de préstamos del libro con el de su padre, si el nuevo elemento posee una cantidad mayor de préstamos, ambos intercambian posiciones, y el proceso continúa hasta restaurar la propiedad del Max Heap: Padre >= Hijos

Eliminación: para eliminar el elemento máximo, se guarda primero el libro ubicado en la raíz, después, el último elemento del arreglo ocupa esa posición y se ejecuta el método Bajar() para recuperar el orden del Heap, donde también se implementó EliminarPorCodigo(), debido a que cuando un libro se elimina del catálogo debe retirarse de todas las estructuras, y en ese caso se busca su posición, se reemplaza por el último elemento y posteriormente se reorganiza el montículo.

Reorganización: cuando se realiza un préstamo aumenta la propiedad VecesPrestado, debido a que esta propiedad determina la prioridad del Max Heap, después del cambio se ejecuta: maxHeap.Reorganizar(codigo)

El método busca el elemento y utiliza las operaciones Subir() y Bajar() para garantizar nuevamente el orden del Heap, cuando el arreglo se llena, el método AumentarCapacidad() crea un arreglo con el doble del tamaño y copia los elementos existentes, de esta forma, la cantidad de libros no queda limitada a la capacidad inicial.

Implementación del Min Heap

El Min Heap utiliza una implementación similar a la del Max Heap, pero cambia la propiedad utilizada como prioridad, en este caso la prioridad es: CopiasDisponibles. Por lo tanto, el libro que posee menor cantidad de copias disponibles se mantiene en: heap[0] y puede obtenerse mediante: VerMinimo()

La propiedad que debe mantenerse es:
Padre $\leq$ Hijos

Inserción: el libro se agrega al final del arreglo y posteriormente se ejecuta Subir(), si el libro posee menos copias disponibles que su padre, se realiza un intercambio, este proceso continúa hasta que la estructura vuelve a cumplir la propiedad del Min Heap.

Eliminación: al eliminar el mínimo, el último elemento del arreglo se mueve a la raíz, después se utiliza Bajar() para compararlo con sus hijos y colocarlo en la posición correspondiente, también se encuentra disponible el método EliminarPorCodigo(), utilizado cuando un libro debe retirarse completamente del sistema.

Reorganización: la cantidad de copias disponibles cambia cuando se registra un préstamo o una devolución, por esta razón, después de modificar dicha propiedad se utiliza: minHeap.Reorganizar(codigo), esto permite que el libro cambie de posición si su nueva disponibilidad altera la prioridad del montículo.

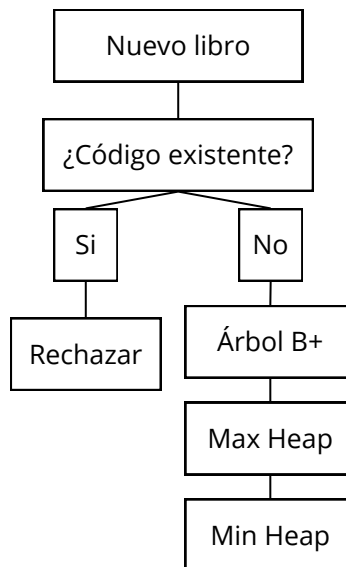
Operaciones principales del sistema

Las operaciones generales se encuentran coordinadas principalmente por la clase Biblioteca, esta clase permite que Program.cs utilice las estructuras sin tener que conocer directamente todos sus detalles internos.

Registro de libros: cuando se registra un libro, primero se busca su código en el Árbol B+, si ya existe un libro con ese código, el registro es rechazado, si el código está disponible, el mismo objeto Libro es insertado en:

Árbol B+
Max Heap
Min Heap

Finalmente se incrementa la cantidad de libros registrados.



Búsqueda de libros

La búsqueda general se realiza utilizando el Árbol B+:

```
return arbol.Buscar(codigo);
```

El código único funciona como clave para localizar el libro.

Aunque los Heaps poseen métodos de búsqueda por código, estos realizan un recorrido del arreglo, debido a que los montículos se encuentran organizados por sus prioridades y no por el código.

Eliminación de libros

Cuando un libro se elimina, primero se comprueba su existencia mediante el Árbol B+. Después se elimina de las tres estructuras:

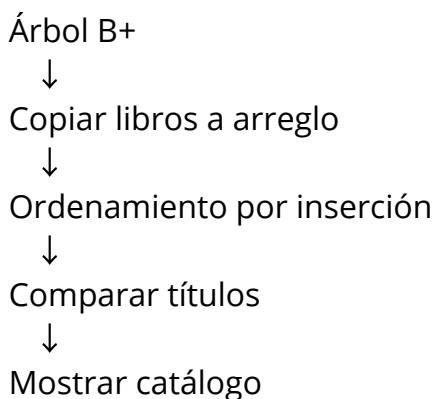
Búsqueda de libros: la búsqueda general se realiza utilizando el Árbol B+:
`return arbol.Buscar(codigo)`, el código único funciona como clave para localizar el libro, aunque los Heaps poseen métodos de búsqueda por código, estos realizan un recorrido del arreglo, debido a que los montículos se encuentran organizados por sus prioridades y no por el código.

Eliminación de libros: cuando un libro se elimina, primero se comprueba su existencia mediante el Árbol B+, después se elimina de las tres estructuras: `arbol.Eliminar(codigo)`; `maxHeap.EliminarPorCodigo(codigo)`, `minHeap.EliminarPorCodigo(codigo)`, esto evita que un libro eliminado permanezca almacenado en alguno de los Heaps.

Registro de préstamo: para registrar un préstamo se busca el libro y se comprueba que tenga al menos una copia disponible, si es posible realizarlo: $\text{CopiasDisponibles} = \text{CopiasDisponibles} - 1$, y $\text{VecesPrestado} = \text{VecesPrestado} + 1$. Después se reorganizan ambos Heaps, el Max Heap se reorganiza porque cambió `VecesPrestado`, mientras que el Min Heap se reorganiza porque cambió `CopiasDisponibles`.

Registro de devolución: para una devolución se verifica primero que el libro exista. También se comprueba que las copias disponibles no sean iguales o mayores a `CopiasTotales`, si la devolución puede realizarse: $\text{CopiasDisponibles} = \text{CopiasDisponibles} + 1$, posteriormente se reorganiza únicamente el Min Heap. No es necesario reorganizar el Max Heap, debido a que `VecesPrestado` no cambia durante una devolución.

Catálogo ordenado por título: el Árbol B+ se encuentra organizado utilizando el código y no el título, por esta razón, para presentar el catálogo ordenado alfabéticamente se copian los libros del Árbol B+ hacia un arreglo temporal utilizando `CopiarLibros()`, luego se aplica un ordenamiento por inserción utilizando la propiedad `Titulo`.



La comparación ignora diferencias entre mayúsculas y minúsculas.

Carga de datos desde archivo CSV

La carga inicial de información se realiza mediante la clase GestorArchivos.
El archivo utilizado es:

Datos/libros.csv

y contiene seis campos:

codigo,titulo,autor,categoria,copiasDisponibles,vecesPrestado

Al iniciar el programa se construye la ruta:

```
string rutaArchivo =Path.Combine("Datos", "libros.csv");
```

Después se llama al método:

```
gestorArchivos.CargarLibros(  
    rutaArchivo,  
    biblioteca  
);
```

El proceso de carga realiza las siguientes acciones:

Verificar existencia del archivo

↓

Leer líneas

↓

Ignorar encabezado

↓

Separar campos mediante coma

↓

Validar valores numéricos

↓

Crear Libro

↓

Biblioteca.RegistrarLibro()

Si una línea contiene datos inválidos, el sistema muestra un mensaje y continúa con el siguiente registro.

También se verifica que no existan códigos duplicados antes de incorporarlos a las estructuras.

Justificación de las estructuras utilizadas

Árbol B+: el Árbol B+ fue utilizado debido a que los libros poseen un código único y una de las operaciones principales del sistema es realizar búsquedas mediante dicho código, la estructura permite mantener las claves organizadas y dirigir la búsqueda a través de nodos internos hasta llegar a las hojas donde se encuentran los libros, además, el enlace entre hojas facilita el recorrido secuencial de los registros.

Max Heap: el Max Heap se utilizó para organizar los libros según la cantidad de veces que han sido prestados, como el elemento de mayor prioridad se mantiene en la raíz, se puede consultar directamente el libro con mayor cantidad de préstamos, y la propiedad utilizada es: VecesPrestado

Min Heap: el Min Heap se utilizó para organizar los libros según la cantidad de copias disponibles, esto permite que el libro con menor disponibilidad se encuentre directamente en la raíz del montículo, la propiedad utilizada es: CopiasDisponibles, el reporte de menor disponibilidad también permite darle una aplicación específica al Min Heap dentro del escenario de biblioteca.

Conclusión

La implementación permitió integrar un Árbol B+, un Max Heap y un Min Heap dentro de un mismo sistema de biblioteca, cada estructura fue utilizada con una función específica y las operaciones fueron coordinadas mediante la clase Biblioteca, manteniendo actualizada la información al registrar, eliminar, prestar o devolver libros. La organización del proyecto en diferentes clases permitió separar la lógica de las estructuras, la gestión de archivos y la interacción con el usuario.